HỌC PHẦN PHẦN MỀM MÃ NGUỒN MỞ · TUẦN 3

Bài tập tổng hợp Chương 3

Sổ điểm lớp học · 90 phút



Nguyễn Dũng
Khoa Công nghệ Thông tin
Trường Đại học Khoa học, Đại học Huế
ndung@husc.edu.vn

Tổng quan bài tập

Xây dựng ứng dụng **Sổ điểm** cho giảng viên:

- **Giao diện web:** xem danh sách sinh viên, xem điểm, tìm kiếm
- **API JSON:** đọc, cập nhật, xoá điểm từng học phần

Phần	Nội dung	Thời gian
0	Khởi tạo dự án, hàm phụ, khung trang	15 phút
1	Giao diện web · Câu 1 – 6	30 phút
2	API JSON · Câu 7 – 8	25 phút
3	Xử lý lỗi · Câu 9	10 phút
4	Kiểm thử, nộp bài	10 phút

Kiến thức sử dụng

Route, converter, `url_for`, method, `request.args`, `escape`, JSON, mã trạng thái, `redirect`, `abort`, `make_response`, `@app.errorhandler`, Flask CLI

Hình thức: cá nhân, **tự viết toàn bộ mã nguồn** từ thư mục trống, được xem tài liệu bài giảng

Yêu cầu chung cho mọi câu

1. Mọi liên kết và mọi header `Location` sinh bằng `url_for`, không viết cứng
2. Mọi dữ liệu chèn vào HTML đều `escape()`, kể cả dữ liệu trong `STUDENTS`
3. JSON hiển thị **tiếng Việt có dấu**
4. Không có view function nào trả về `None`
5. **Commit ít nhất 4 lần**, sau mỗi Phần 0, 1, 2, 3

Trừ 0,5 điểm

cho mỗi loại vi phạm:

- viết cứng URL
- không escape dữ liệu
- view function trả `None`
- ít hơn 4 commit, hoặc commit `.venv/`

Lịch sử commit là căn cứ xác nhận bài do sinh viên **tự làm**.

Phần 0. Khởi tạo dự án

Môi trường ảo · dữ liệu · hàm phụ · khung trang · 15 phút

Câu 0.1 – 0.2. Dự án và dữ liệu mẫu 5 phút

```
mkdir -p chuong_03/tong_hop
cd chuong_03/tong_hop
python3 -m venv .venv
source .venv/bin/activate
pip install flask
pip freeze > requirements.txt
```

- `.gitignore`: bỏ qua `.venv/`, `__pycache__/`
- `sodiem.py`: tạo `app`, cấu hình JSON tiếng Việt có dấu

```
flask --app sodiem run --debug --port 8000
```

Dữ liệu mẫu, **chỉ phần này** được sao chép từ đề:

```
STUDENTS = {
    "23T1020001": {"name": "Nguyễn Văn An", "lop": "K47A",
      "scores": {"PMMNM": 8.5, "CSDL": 7.0, "MMT": 9.0}},
    "23T1020002": {"name": "Trần Thị Bình", "lop": "K47A",
      "scores": {"PMMNM": 6.0, "CSDL": 5.5, "MMT": 7.0}},
    "23T1020003": {"name": "Lê Hoàng Cường", "lop": "K47B",
      "scores": {"PMMNM": 9.5, "CSDL": 9.0}},
    "23T1020004": {"name": "Phạm Minh Dũng", "lop": "K47B",
      "scores": {"PMMNM": 4.0, "CSDL": 3.5, "MMT": 5.0}},
    "23T1020005": {"name": "Hoàng Thu Hà", "lop": "K47A",
      "scores": {}},
    "23T1020006": {"name": "Võ Quốc Khánh", "lop": "K47C",
      "scores": {"PMMNM": 7.5, "MMT": 8.0}},
}
```

Câu 0.3 – 0.4. Hàm phụ và khung trang 10 phút

Hàm	Đặc tả
`average(scores)`	Trung bình cộng, **làm tròn 2 chữ số**; dict rỗng → `None`
`rank(avg)`	`None` → "Chưa có điểm"; ≥ 8,5 Giỏi; ≥ 7,0 Khá; ≥ 5,0 Trung bình; còn lại Yếu
`student_summary(mssv)`	Dict `{"mssv", "name", "lop", "scores", "average", "rank"}`
`layout(title, body)`	Trang HTML hoàn chỉnh; `<title>` dạng `… - Sổ điểm` (escape `title`); menu **Trang chủ · Sinh viên · Tìm kiếm** sinh bằng `url_for`; `body` chèn nguyên văn

`student_summary("23T1020001")` → `average` **8.17**, `rank` **Khá**
`student_summary("23T1020005")` → `average` **None**, `rank` **Chưa có điểm**

`url_for` chỉ sinh được URL của endpoint **đã khai báo**: tạo trước các route `/`, `/students`, `/search` tạm thời, hoàn thiện ở Phần 1.

Phần 1. Giao diện web

Câu 1 – 6 · 30 phút

Câu 1 – 2. Trang chủ và danh sách 13 phút

Câu 1. `/` : tổng số sinh viên, số lớp (không trùng), liên kết đến `/students` và `/api/students`

Câu 2. `/students`

- Bảng: **MSSV** (liên kết đến trang chi tiết), **Họ tên**, **Lớp**, **Điểm TB**, **Xếp loại**
- Chưa có điểm: cột Điểm TB hiển thị `–`
- Lọc `?lop=K47A` , **không phân biệt hoa thường**
- Thanh lọc `Tất cả | K47A | K47B | K47C` lấy từ dữ liệu, **không viết cứng**
- Không có kết quả: "Không có sinh viên phù hợp."

Gợi ý

```
url_for("student_list", lop="K47A")
# /students?lop=K47A
```

Tham số không khớp biến trên URL thì `url_for` đưa vào **query string**.

Câu 3 – 4. Chi tiết và link rút gọn 8 phút

Câu 3. `/students/<mssv>`

- Họ tên, MSSV, điểm TB, xếp loại
- Lớp: liên kết về danh sách **đã lọc** theo lớp đó
- Bảng điểm từng học phần
- MSSV không tồn tại:

```
abort(404, description=
      f"Không có sinh viên với MSSV = {mssv}.")
```

Câu 4. `/sv/<mssv>`

Chuyển hướng tới `/students/<mssv>` với mã **301**

```
curl -i http://127.0.0.1:8000/sv/23T1020001
```

```
HTTP/1.1 301 MOVED PERMANENTLY
Location: /students/23T1020001
```

Câu 5. Xuất bảng điểm CSV 4 phút

`/students/<mssv>/export`: dùng `make_response` để trình duyệt **tải file xuống** thay vì hiển thị

Header	Giá trị
`Content-Type`	`text/csv; charset=utf-8`
`Content-Disposition`	`attachment; filename=diem_<mssv>.csv`

MSSV không tồn tại: **404**

Trang chi tiết (Câu 3) có liên kết **"Tải bảng điểm (CSV)"** và hiển thị link rút gọn của Câu 4.

Nội dung file `diem_23T1020001.csv`:

```
hoc_phan,diem
PMMNM,8.5
CSDL,7.0
MMT,9.0
```

Câu 6. Tìm kiếm an toàn 5 phút

`/search?q=...`

- Form `GET` có ô nhập, **giữ lại từ khoá** đã nhập trong ô
- Tìm sinh viên có **họ tên chứa** từ khoá (không phân biệt hoa thường) **hoặc MSSV chứa** từ khoá
- Hiển thị: Tìm thấy N kết quả cho "...", kèm danh sách liên kết

Kiểm tra XSS

```
/search?q=<script>alert(1)</script>
/search?q="><script>alert(1)</script>
```

Trình duyệt **không** được hiện hộp thoại với cả hai URL.

Lưu ý: từ khoá xuất hiện ở **hai chỗ**, trong kết quả và trong thuộc tính `value` của ô nhập.

Phần 2. API JSON

Câu 7 – 8 · 25 phút

Câu 7. API đọc dữ liệu 8 phút

Request	Response
`GET /api/students`	List các dict do `student_summary()` trả về
`GET /api/students?lop=K47A`	Chỉ sinh viên lớp K47A, không phân biệt hoa thường
`GET /api/students?min_avg=7`	Chỉ sinh viên có điểm TB ≥ 7, bỏ qua sinh viên chưa có điểm
`GET /api/students?min_avg=abc`	**400**
`GET /api/students/<mssv>`	Dict của một sinh viên; không tồn tại → **404**

Hai tham số `lop` và `min_avg` dùng được **cùng lúc**.

`request.args.get("min_avg", type=float)` trả `None` cả khi **thiếu** tham số lẫn khi tham số **sai kiểu**. Cần phân biệt hai trường hợp này.

Câu 8. Quản lý điểm một học phần 17 phút

Một route duy nhất `/api/students/<mssv>/scores/<course>`. Mã học phần không phân biệt hoa thường, luôn lưu dạng chữ hoa (`web` → `WEB`). Điểm **0** là điểm hợp lệ.

Request	Hành vi	Response
`GET`	Xem điểm	`{"mssv", "course", "score"}`
`PUT ?score=9`, học phần **chưa có**	Thêm điểm	**201** + header `Location` trỏ về chính URL này; body thêm `"average"`
`PUT ?score=7.5`, học phần **đã có**	Sửa điểm	**200**, body như trên
`PUT` thiếu `score`, sai kiểu, ngoài [0, 10]		**400**
`DELETE`	Xoá điểm	**204**, body rỗng
`GET` / `DELETE` khi học phần chưa có điểm		**404**
Mọi method, MSSV không tồn tại		**404**
`POST`	Không hỗ trợ	**405**

Phần 3 – 4. Xử lý lỗi và kiểm thử

Câu 9 · 10 phút · Kiểm thử, nộp bài · 10 phút

Câu 9. Trang lỗi thống nhất 10 phút

Viết **một** hàm xử lý chung cho lỗi **400, 404, 405**

- URL bắt đầu bằng `/api/` → JSON: `{"error": "<tiêu đề>", "detail": "<error.description>"}`
- URL khác → trang HTML dùng `layout()`: mã lỗi, tiêu đề, mô tả
- Luôn trả **đúng mã lỗi**, không để Flask trả `200`

Mã	Tiêu đề
400	Dữ liệu không hợp lệ
404	Không tìm thấy
405	Phương thức không được hỗ trợ

Gợi ý:

đặt nhiều decorator chồng lên nhau

```
@app.errorhandler(400)
@app.errorhandler(404)
@app.errorhandler(405)
def handle_error(error):
    ...
```

Câu hỏi:

vì sao dùng được `request` trong hàm này dù nó không phải view function?

Phần 4. Kiểm thử bằng `curl` 10 phút

```
B=http://127.0.0.1:8000
curl -i $B/sv/23T1020001                          # 301
curl -i $B/students/23T1020001/export             # CSV
curl    "$B/api/students?lop=k47a&min_avg=7"
curl -i "$B/api/students?min_avg=abc"             # 400
curl -i $B/api/students/999                       # 404
S=$B/api/students/23T1020005/scores
curl -i -X PUT    "$S/web?score=9"                # 201
curl    -X PUT    "$S/WEB?score=7.5"              # 200
curl -i -X PUT    "$S/WEB?score=11"               # 400
curl -i -X DELETE $S/WEB                          # 204
curl -i -X POST   $S/WEB                          # 405 JSON
curl -i -X POST   $B/students                     # 405 HTML
```

Ghi vào `README.md`:

1. Kết quả `flask --app sodiem routes`, phải có **10 dòng** (kể cả `static`)
2. Dòng trạng thái và body (hoặc header quan trọng) của từng lệnh `curl`
3. Trả lời ngắn:
   - Vì sao Câu 4 dùng **301** còn Câu 8 trả **201** kèm `Location`?
   - Thêm điểm cho `23T1020005` rồi khởi động lại server, điểm đó còn không? Vì sao?

> **Nộp:** thư mục `chuong_03/tong_hop/`: `sodiem.py`, `requirements.txt`, `.gitignore`, `README.md`, ≥ 4 commit

Thang điểm · 10 điểm

Nội dung	Điểm
Phần 0. Khởi tạo, hàm phụ, khung trang	1,0
Câu 1 – 2. Trang chủ, danh sách, lọc	1,5
Câu 3. Chi tiết, 404	0,5
Câu 4. Chuyển hướng 301	0,5
Câu 5. Xuất CSV	0,5
Câu 6. Tìm kiếm an toàn	1,0
Câu 7. API đọc	1,5
Câu 8. API điểm	2,0
Câu 9. Xử lý lỗi	1,0
Phần 4. README, câu hỏi	0,5

Tiêu chí then chốt

- Câu 7: phân biệt **thiếu** và **sai kiểu** `min_avg`
- Câu 8: đúng cả 8 trường hợp, đặc biệt **201 / 200**, **204**, **405**
- Câu 9: JSON cho `/api/`, HTML cho trang khác, đúng mã lỗi

Trừ 0,5

mỗi loại vi phạm: viết cứng URL · không escape · view trả `None` · ít hơn 4 commit hoặc commit `.venv/`

Bắt đầu làm bài

90 phút · bắt đầu từ thư mục trống

Nguyễn Dũng
ndung@husc.edu.vn
Khoa Công nghệ Thông tin, Trường Đại học Khoa học, Đại học Huế